# PROJECT JACK: TOP 15 ARCHITECTURAL FEATURES

*Ranked Security, Performance, and Cognitive Integrity Vector Analysis*

| | |
|---|---|
| **Date:** | June 11, 2026 |
| **Classification:** | Technical Reference |
| **Document ID:** | JACK-ARCH-REV-015 |
| **Target Stack:** | Deterministic Chassis & Prompt Optimization Layer |

The following registry profiles and ranks the top 15 architectural mechanisms of Project Jack. Ranks are determined by evaluating each vector's cross-functional contribution to defensive containment, multi-modal fail-safes, context resilience, and token economic efficiency. This hierarchy emphasizes physical, deterministic isolation frameworks over soft, probabilistic linguistic controls.

## ARCHITECTURAL REGISTRY & RANKINGS

### #1 StreamingIRQ & 3-Tier Canary Tripwires (Pillar 7)

PILLAR/LAYER: STREAM BOUNDARY INNOVATION: ★★★★★ | EFFECTIVENESS: ★★★★★ | PRACTICALITY: ★★★★★

**TECHNICAL SPECIFICATION**

Real-time sliding window (256-char) token quarantine prior to downstream output; severs stream (SovereignInterrupt) on tripwire match.

**WHY IT RANKS #1**

Holds output in a physical quarantine window rather than relying on prompt-level compliance. By compiling Tier A (Auto-Vault), Tier B (Custom YAML), and Tier C (Dynamic Session) canaries directly into the active validator, credential exfiltration is intercepted and stopped mid-token.

### #2 Transactional Write-Ahead Log (WAL) & Ghost Ledger (Pillar 3)

PILLAR/LAYER: STATE COMMIT & COMMAND EXECUTION
INNOVATION: ★★★★★ | EFFECTIVENESS: ★★★★★ | PRACTICALITY: ★★★★★

**TECHNICAL SPECIFICATION**

Append-only transactional JSONL ledger (.jack/trajectory_wal.jsonl) capturing step executions with immediate os.fsync() flushes. It logs monotonically increasing step_ids, container digests, and execution hashes of the safe, DLP-scrubbed canonical arguments.

**WHY IT RANKS #2**

Eliminates the risk of double-executing non-idempotent real-world actions during recovery loops. By verifying the cryptographic lineage of each step, the Chassis proves prior execution without trusting the model's memory, converting trajectory resumption into an immutable, crash-safe transaction ledger.

### #3 Zero-Network Math Sandbox (Phase 30 Math Hardening)

PILLAR/LAYER: SYMBOLIC SANDBOX/TOOL EXECUTION

INNOVATION: ★★★★★ | EFFECTIVENESS: ★★★★★ | PRACTICALITY: ★★★★★

**TECHNICAL SPECIFICATION**

Containerized, non-networked math environment pre-compiled with Z3 SMT and SymPy. Resource limits (mem_limit="512m", 50% CPU) prevent denial-of-service or container-escape exploits.

**WHY IT RANKS #3**

Rejects risky host eval() or exec() loops. The Chassis handles complex algebraic and constraint logic inside an isolated container with network_mode="none", injecting symbolic solvers (Z3) to guarantee mathematically correct results offline.

### #4 TAS & Forced Deliberation (Pillar 2)

PILLAR/LAYER: ADVERSARIAL REVIEW LOOP INNOVATION: ★★★★★ | EFFECTIVENESS: ★★★★★ | PRACTICALITY: ★★★★★

**TECHNICAL SPECIFICATION**

Decentralized cognitive loop: Architect (Thesis), Sage (Antithesis), Synthesizer (Synthesis). Preemptively enforces "Think Max" reasoning constraints (alternatives, failure modes, rejected hypotheses) on the Architect's prompt.

**WHY IT RANKS #4**

Converts the highly expensive post-hoc Confirmation-Violence retries into a structured, proactive forward pass. By obligating the Engine to document its complete deliberation process up front, we significantly reduce retry loop overhead and token waste.

### #5 XML Expanded Knowledge Anchoring (Sovereign Policy Engine / Pillar 5)

PILLAR/LAYER: MEMORY & RETRIEVAL LAYER (PILLAR V / LIBRARIAN) INNOVATION: ★★★★★ | EFFECTIVENESS: ★★★★★ | PRACTICALITY: ★★★★★

**TECHNICAL SPECIFICATION**

Custom-parsed XML-schema format (<pattern>, <why>, <how>, <pitfall>) compiled within Expanded_Knowledge.md. It serves as a live, declarative runtime policy console. On boot, individual patterns are extracted as atomic cards, indexed securely by the FileInterceptor, and semantically retrieved using the Manager's JSON execution plan.

**WHY IT RANKS #5**

Directly elevates the user from a passive prompter to the absolute legislator of Jack's cognitive boundaries. By appending structured markdown cards, users can dynamically re-program Jack's runtime security, style, and mathematical invariants without touching the codebase. Most critically, it leverages the <pitfall> tag as an explicit restrictive axis, mathematically pruning the local model's attention heads inside the HotContext to block unsafe execution paths before a single token is synthesized.

### #6 Periodic Context Defragmentation (HotContext / Pillar 6)

PILLAR/LAYER: MEMORY & CONTEXT MANAGEMENT INNOVATION: ★★★★★ | EFFECTIVENESS: ★★★★★ | PRACTICALITY: ★★★★★

**TECHNICAL SPECIFICATION**

Token watermark monitoring. At 75% capacity, the Chassis intercepts and commands the Librarian (Pillar 6) to compress the oldest 50% of context chunks into a high-density, system-trusted summary, discarding verbose raw inputs.

**WHY IT RANKS #6**

Prevents "Lost in the Middle" recall degradation and memory exhaustion (OOMs) during long-horizon tasks. Implements a robust constructor Type-Coercion Guard in HotContext to immunize the buffer against mock object bleed-through during test execution.

### #7 SSRF Web Navigation Firewalls (DNS Rebinding Protection)

PILLAR/LAYER: WEB NAVIGATION/TOOL BOUNDARY INNOVATION: ★★★★★ | EFFECTIVENESS: ★★★★★ | PRACTICALITY: ★★★★★

**TECHNICAL SPECIFICATION**

Custom DNSRebindingSafeHTTPSHandler and DNSRebindingSafeHTTPHandler subclasses for urllib.request. Resolves hostnames exactly once, validates against loopback, private, multicast, and AWS metadata (169.254.169.254) IP ranges, and binds connections directly to the pre-verified IP.

**WHY IT RANKS #7**

Standard agents calling simple request libraries are highly vulnerable to SSRF and DNS Rebinding attacks. Jack's custom handlers completely eliminate this network escape vector by forcing connection sockets to lock onto pre-audited IPs, bypassing subsequent connection-time DNS lookups entirely.

### #8 Hardened Ingestion Gate: Inodes & Descriptor Pinning (Pillar 4)

PILLAR/LAYER: EYES INGESTION BOUNDARY INNOVATION: ★★★★★ | EFFECTIVENESS: ★★★★★ | PRACTICALITY: ★★★★★

**TECHNICAL SPECIFICATION**

Resolves the physical st_dev/st_ino tree on open file descriptors, completely blocking symlinks, hardlinks, and TOCTOU directory escapes.

**WHY IT RANKS #8**

Standard path-based string matching is vulnerable to symbolic link routing or hardlinks created outside protected directories (such as vault/ or jack/). Pinning the descriptor at the inode level unmasks the true file state before reading begins.

### #9 Deterministic Router & Learned Verb Allowlist

PILLAR/LAYER: ROUTING LAYER INNOVATION: ★★★★★ | EFFECTIVENESS: ★★★★★ | PRACTICALITY: ★★★★★

**TECHNICAL SPECIFICATION**

Non-ML intent classification using explicit token-length rules. Safe read-only actions are appended to a signed allowlist in the Librarian, bypassing planning friction on repeat tasks.

**WHY IT RANKS #9**

Eliminate expensive, non-deterministic classifiers. By utilizing deterministic token math and a verified verb allowlist, Jack cuts operational API costs by 60% while maintaining absolute routing consistency.

### #10 Contract Validator Pre-Flight Checks (Pillar 1)

PILLAR/LAYER: LEGAL/POLICY LAYER INNOVATION: ★★★☆☆ | EFFECTIVENESS: ★★★★★ | PRACTICALITY: ★★★★★

**TECHNICAL SPECIFICATION**

Preventive syntax and command blacklisting compiled as RE2-backed linear-time regular expressions.

**WHY IT RANKS #10**

Audits proposed plans prior to container creation, rejecting directory traversals, blacklisted commands, and obfuscated base64 scripts before the Muscle can execute them.

### #11 Chunked Sliding-Window DLP Audit (High IRQ)

PILLAR/LAYER: OUTBOUND QUARANTINE AUDIT INNOVATION: ★★★★★ | EFFECTIVENESS: ★★★★★ | PRACTICALITY: ★★★★★

**TECHNICAL SPECIFICATION**

8KB sliding-chunk file scanning with a mandatory 256-byte overlap buffer to catch split-credentials (e.g., AWS access keys) across chunk boundaries.

**WHY IT RANKS #11**

Standard file scanners load the entire generated artifact into memory, risking OOM exploits. Jack stream-audits generated files in bounded, memory-efficient chunks, shredding the quarantine buffer instantly on violation.

### #12 Departmentalized Cognition via 7 Locked Pillars

PILLAR/LAYER: COGNITIVE LAYER INNOVATION: ★★★★☆ | EFFECTIVENESS: ★★★★☆ | PRACTICALITY: ★★★★★

**TECHNICAL SPECIFICATION**

Rigid separation of planning, auditing, memory, and synthesis into isolated pillars bound by specific, immutable mandates verified at boot.

**WHY IT RANKS #12**

Prevents role collapse (a single model planning, executing, and reviewing itself). If one pillar is compromised by prompt injection, the strict boundaries set by the Chassis prevent the compromise from spreading.

### #13 Immutable Reference Data Volumes (Phase 30)

PILLAR/LAYER: SANDBOX INGEST INNOVATION: ★★★★★ | EFFECTIVENESS: ★★★★★ | PRACTICALITY: ★★★★★

**TECHNICAL SPECIFICATION**

Offline reference dataset bind-mounting (/datasets:ro) into the sandbox container.

**WHY IT RANKS #13**

Since the sandbox operates completely offline, the Engine cannot fetch reference data. Jack resolves this securely by mounting read-only databases and reference tables, allowing complex calculations with zero risk of data exfiltration or SSRF.

### #14 Vault Secrets-at-Rest with In-Memory Bytearrays

PILLAR/LAYER: CREDENTIALS LAYER INNOVATION: ★★★★★ | EFFECTIVENESS: ★★★★★ | PRACTICALITY: ★★★★★

**TECHNICAL SPECIFICATION**

Vault secrets stored as mutable bytearray buffers in RAM, physically overwritten with null bytes after use.

**WHY IT RANKS #14**

Decrypted keys exist in RAM only for the duration of the request boundary, preventing memory-scraping exploits from harvesting long-lived plain-text credentials.

### #15 Telemetry Kill Switch

PILLAR/LAYER: TELEMETRY LAYER INNOVATION: ★★★☆☆ | EFFECTIVENESS: ★★★★★ | PRACTICALITY: ★★★★☆

**TECHNICAL SPECIFICATION**

Global control flag that instantly redacts exception messages to "Redacted local error." and zeroes out metadata tracing payloads.

**WHY IT RANKS #15**

Critical for enterprise codebases. Standard agents leak local variables and file context during remote crash reporting. With telemetry disabled, Jack guarantees private local data never escapes the host.